



Keshav Mahavidyalaya

University of Delhi

Department of Computer Science

Information Security

Practical File

Name – Ankit Kumar
Roll No. – 16005, Sem- 8
Exam Roll No. – 22035570007
Course – B.Sc. (Hons.) Computer Science

Practical 1: Network Tools

Aim: Demonstrate the use of network diagnostic tools: ping, ipconfig, ifconfig, tracert, arp, netstat, whois

Tools Required: Windows / Linux terminal, internet connection

1.1 ping

ping tests reachability of a host by sending ICMP echo request packets and measuring round-trip time.

Steps:

- Open Command Prompt (Windows) or Terminal (Linux/macOS).
- Type the ping command followed by a hostname or IP.
- Observe packet statistics: sent, received, lost, and average RTT.

Commands:

```
ping google.com
ping 8.8.8.8
ping -c 4 google.com
```

Sample Output:

```
C:\Users\nitis>ping google.com

Pinging google.com [2404:6800:4002:82a::200e] with 32 bytes of data:
Reply from 2404:6800:4002:82a::200e: time=34ms
Reply from 2404:6800:4002:82a::200e: time=164ms
Reply from 2404:6800:4002:82a::200e: time=45ms
Reply from 2404:6800:4002:82a::200e: time=88ms

Ping statistics for 2404:6800:4002:82a::200e:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 34ms, Maximum = 164ms, Average = 82ms
```

1.2 ipconfig (Windows)

ipconfig displays IP address, subnet mask, and default gateway for all network adapters.

Steps:

- Open Command Prompt.
- Run the command and note the IPv4 Address, Subnet Mask, and Default Gateway.

Commands:

```
ipconfig
ipconfig /all           # detailed - shows MAC, DHCP server, DNS
ipconfig /release       # release current IP lease
ipconfig /renew          # obtain new IP from DHCP server
ipconfig /flushdns       # clear DNS resolver cache
```

```

C:\Users\nitis>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . : kmv.in

Ethernet adapter Ethernet 2:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::36da:eabd:530d:95a0%5
    IPv4 Address. . . . . : 192.168.56.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Wireless LAN adapter Local Area Connection* 3:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 4:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::c9e6:6543:3240:c4aa%24
    IPv4 Address. . . . . : 192.168.137.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

```

1.3 ifconfig (Linux/macOS)

ifconfig is the Linux/macOS equivalent of ipconfig.

Commands:

```

ifconfig
ifconfig eth0          # show details of interface eth0
ip addr show           # modern replacement for ifconfig

```

1.4 tracert / traceroute

Traces the route packets take to reach a destination, showing each hop (router) along the path.

Steps:

- Run the command with a destination hostname.
- Each line shows one hop: hop number, round-trip times (×3), and the router IP.
- An asterisk (*) means the router did not respond within the timeout.

Commands:

```

tracert google.com      # Windows
traceroute google.com   # Linux/macOS
traceroute -n google.com # Linux - skip DNS resolution

```

```

C:\Users\nitis>tracert google.com

Tracing route to google.com [2404:6800:4002:82a::200e]
over a maximum of 30 hops:

  1    53 ms    4 ms    1 ms    2409:4050:e30:7550::e4
  2    *        *        *        Request timed out.
  3    54 ms    36 ms    37 ms    2405:200:330:eeee:20::822
  4    61 ms    203 ms   39 ms    2405:200:801:300::e11
  5    *        *        *        Request timed out.
  6    *        *        *        Request timed out.
  7    *        *        *        Request timed out.
  8    *        *        *        Request timed out.
  9    46 ms    92 ms    135 ms   2001:4860:1:1::17ae
 10    39 ms    56 ms    42 ms    2001:4860:0:1::78d3
 11    52 ms    80 ms    49 ms    2001:4860:0:1::539f
 12    80 ms    186 ms   135 ms   tzdelb-ay-in-x0e.1e100.net [2404:6800:4002:82a::200e]

Trace complete.

```

1.5 arp

ARP (Address Resolution Protocol) maps IP addresses to MAC addresses on the local network.

Commands:

```

arp -a                # display ARP cache (all platforms)
arp -a 192.168.1.1    # show entry for specific IP
arp -d 192.168.1.1    # delete an entry (Windows, run as admin)
arp -s 192.168.1.50 AA-BB-CC-DD-EE-FF # add static entry (Windows)

```

1.6 netstat

netstat displays active network connections, listening ports, routing table, and interface statistics.

Commands:

```

netstat -a            # all connections and listening ports
netstat -n            # numeric addresses (skip DNS lookup)
netstat -b            # Windows - show executable owning each
connection
netstat -r            # display routing table
netstat -s            # per-protocol statistics
netstat -tulnp        # Linux - TCP/UDP listening ports with PID

```

1.7 whois

whois queries WHOIS databases to retrieve domain registration information, owner details, and name servers.

Steps:

- whois is built-in on Linux. On Windows, download from Sysinternals or use an online tool.
- Run the command with a domain name.

- Observe the registrar, registration/expiry dates, and name servers.

Commands:

```
whois google.com           # Linux/macOS
whois 8.8.8.8              # reverse lookup for an IP
# Windows (Sysinternals whois):
whois.exe google.com
```

Practical 2: Password Cracking Tools

Aim: Verify password strength using John the Ripper and Ophcrack

Tools Required: Kali Linux (or any Linux distro), John the Ripper, Ophcrack, sample password hashes

2.1 John the Ripper

John the Ripper (JtR) is an open-source password security auditing and recovery tool. It supports hundreds of hash types.

Installation:

```
| sudo apt update && sudo apt install john -y           # Debian/Ubuntu/Kali
```

Step 1 – Create a test password hash file:

```
| # Create a test user entry (Linux shadow format)
echo 'testuser:$(openssl passwd -6 password123)' > test_shadow.txt
cat test_shadow.txt
```

Step 2 – Identify the hash format:

```
| john --list=formats | grep -i sha
```

Step 3 – Run a wordlist attack (dictionary attack):

```
| john --wordlist=/usr/share/wordlists/rockyou.txt test_shadow.txt
```

Step 4 – Show cracked passwords:

```
| john --show test_shadow.txt
```

Step 5 – Brute-force attack with incremental mode:

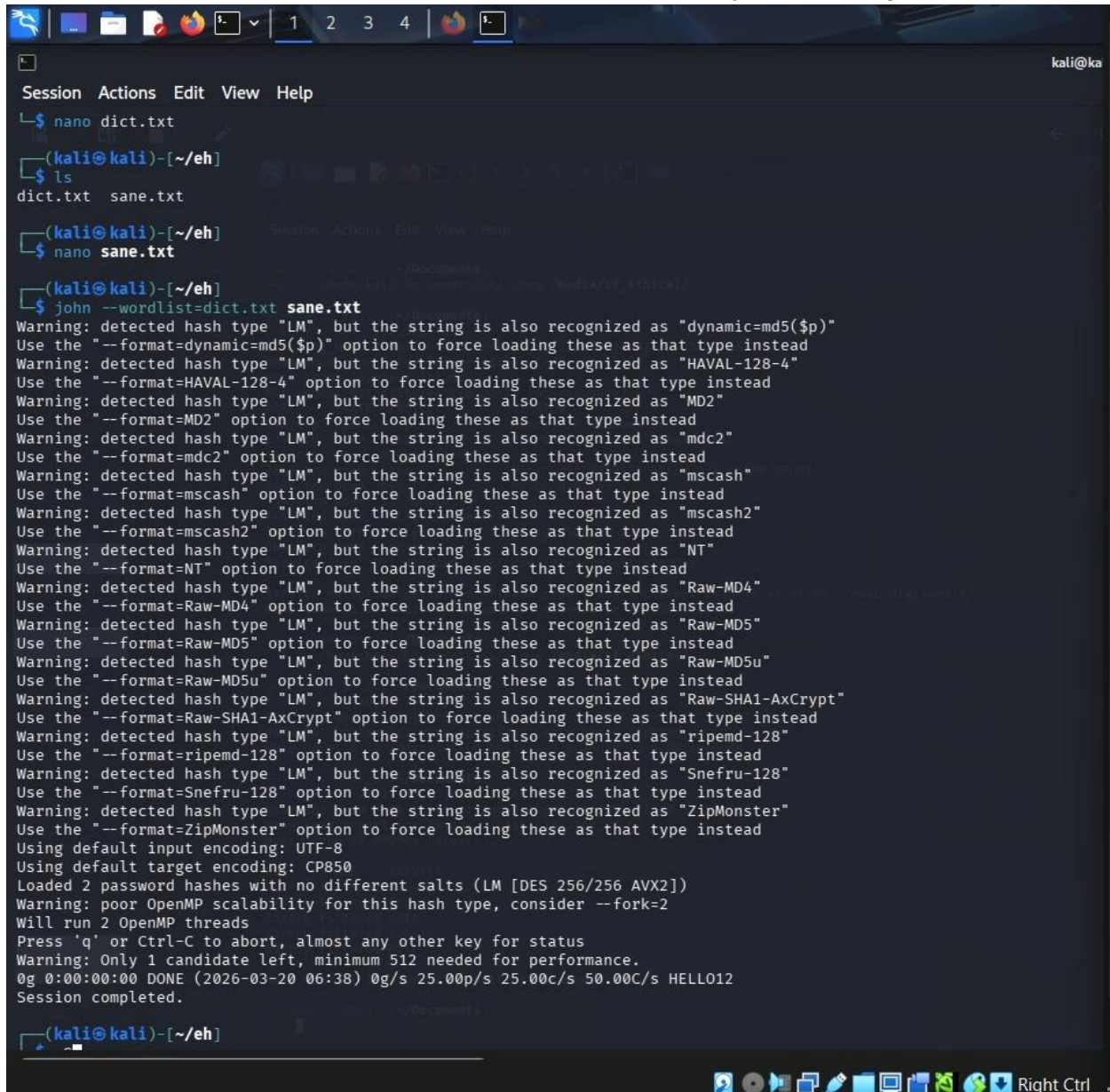
```
| john --incremental test_shadow.txt
```

Step 6 – Crack MD5 hash:

```
| # Generate MD5 hash of a password
echo -n 'password123' | md5sum > md5hash.txt
# Format for JtR: username:hash
echo 'user1:$(cat md5hash.txt | cut -d' ' ' -f1)' > md5_test.txt
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt
md5_test.txt
```

Observations:

- Weak passwords (e.g., password123, abc123) are cracked within seconds using a wordlist.
- Strong passwords (long, mixed case, symbols, numbers) take significantly longer.
- Incremental mode tries all character combinations – very slow for long passwords.



```
kali@kali:~$ nano dict.txt
(kali@kali)-[~/eh]
$ ls
dict.txt  sane.txt
(kali@kali)-[~/eh]
$ nano sane.txt
(kali@kali)-[~/eh]
$ john --wordlist=dict.txt sane.txt
Warning: detected hash type "LM", but the string is also recognized as "dynamic-md5($p)"
Use the "--format=dynamic-md5($p)" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "HAVAL-128-4"
Use the "--format=HAVAL-128-4" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "MD2"
Use the "--format=MD2" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "mdc2"
Use the "--format=mdc2" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "mscash"
Use the "--format=mscash" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "mscash2"
Use the "--format=mscash2" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "NT"
Use the "--format=NT" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "Raw-MD4"
Use the "--format=Raw-MD4" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "Raw-MD5"
Use the "--format=Raw-MD5" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "Raw-MD5u"
Use the "--format=Raw-MD5u" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "Raw-SHA1-AxCrypt"
Use the "--format=Raw-SHA1-AxCrypt" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "ripemd-128"
Use the "--format=ripemd-128" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "Snefru-128"
Use the "--format=Snefru-128" option to force loading these as that type instead
Warning: detected hash type "LM", but the string is also recognized as "ZipMonster"
Use the "--format=ZipMonster" option to force loading these as that type instead
Using default input encoding: UTF-8
Using default target encoding: CP850
Loaded 2 password hashes with no different salts (LM [DES 256/256 AVX2])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 1 candidate left, minimum 512 needed for performance.
0g 0:00:00:00 DONE (2026-03-20 06:38) 0g/s 25.00p/s 25.00c/s 50.00C/s HELLO12
Session completed.
(kali@kali)-[~/eh]
```

Practical 3: Network Scanning with Nmap / Zenmap

Aim: Analyze a remote machine to discover open ports, services, and OS information

Tools Required: Kali Linux, Nmap, Zenmap (GUI front-end for Nmap), target machine (with permission)

3.1 Installation

```
| sudo apt update && sudo apt install nmap zenmap -y
```

3.2 Basic Scans

Ping scan (host discovery):

```
| nmap -sn 192.168.1.0/24
```

TCP SYN scan (stealth scan – requires root):

```
| sudo nmap -sS 192.168.1.1
```

Full TCP connect scan:

```
| nmap -sT 192.168.1.1
```

UDP scan:

```
| sudo nmap -sU 192.168.1.1
```

3.3 Port & Service Detection

```
| nmap -p 1-1024 192.168.1.1      # scan specific port range
| nmap -p- 192.168.1.1           # scan all 65535 ports
| nmap -sV 192.168.1.1           # detect service versions
| nmap -sV -p 80,443,22 192.168.1.1 # version detection on specific ports
```

3.4 OS Detection

```
| sudo nmap -O 192.168.1.1
| sudo nmap -A 192.168.1.1      # aggressive: OS + version + scripts +
| traceroute
```

3.5 Script Scanning (NSE)

```
| nmap --script=default 192.168.1.1
| nmap --script=vuln 192.168.1.1      # vulnerability scan
| nmap --script=http-headers 192.168.1.1 # HTTP headers
| nmap --script=ftp-anon 192.168.1.1  # check for anonymous FTP
```

3.6 Output Options

```
| nmap -oN scan_result.txt 192.168.1.1 # normal text output
| nmap -oX scan_result.xml 192.168.1.1 # XML output
| nmap -oG scan_result.gnmap 192.168.1.1 # grepable output
```

Observations:

- Open ports reveal running services; unnecessary open ports increase attack surface.

- Service version detection helps identify outdated, vulnerable software.
- Always obtain written permission before scanning any network or machine.

```
(kali㉿kali)-[~/Documents]
└─$ nmap -p 20,21,25,53,80
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-10 13:57 EST
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.06 seconds

(kali㉿kali)-[~/Documents]
└─$ nmap -p 20,21,25,53,80 192.168.43.5
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-10 13:57 EST
Nmap scan report for Insane_nitish (192.168.43.5)
Host is up (0.00090s latency).

PORT      STATE      SERVICE
20/tcp    filtered  ftp-data
21/tcp    filtered  ftp
25/tcp    filtered  smtp
53/tcp    filtered  domain
80/tcp    filtered  http

Nmap done: 1 IP address (1 host up) scanned in 1.70 seconds
```

•

```
└─$ nmap -sV 192.168.43.5
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-10 13:52 EST
Nmap scan report for Insane_nitish (192.168.43.5)
Host is up (0.0044s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds?
2869/tcp   open  http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
3306/tcp   open  mysql        MySQL 8.0.36
4343/tcp   open  ssl/unicall?
4449/tcp   open  ssl/privatewire?
5432/tcp   open  postgresql    PostgreSQL DB 9.6.0 or later
6646/tcp   open  unknown
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 184.96 seconds
```

•

```
(kali㉿kali)-[~/Documents]
$ nmap -sS 192.168.43.5
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-10 13:50 EST
Nmap scan report for Insane_nitish (192.168.43.5)
Host is up (0.0037s latency).
Not shown: 991 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
2869/tcp   open  icslap
3306/tcp   open  mysql
4343/tcp   open  unicall
4449/tcp   open  privatewire
5432/tcp   open  postgresql
6646/tcp   open  unknown

Nmap done: 1 IP address (1 host up) scanned in 5.68 seconds
```

Practical 4: Burp Suite – Capture and Modify HTTP Messages

Aim: Use Burp Suite's proxy to intercept, inspect, and modify HTTP/HTTPS traffic

Tools Required: Burp Suite Community Edition, Firefox browser, target web application (e.g., DVWA or testfire.net)

4.1 Installation & Launch

```
# Download Burp Suite Community Edition from portswigger.net
# OR on Kali Linux:
sudo apt install burpsuite -y
burpsuite &
```

4.2 Configure Browser Proxy

Steps:

- In Firefox, open Settings → Network Settings → Manual proxy configuration.
- Set HTTP Proxy: 127.0.0.1, Port: 8080.
- Check 'Also use this proxy for HTTPS'.
- Click OK.

4.3 Install Burp CA Certificate (for HTTPS)

Steps:

- With Burp running and browser proxy set, visit: <http://burp>
- Click CA Certificate to download the certificate.
- In Firefox: Settings → Privacy & Security → Certificates → View Certificates.
- Click Import and select the downloaded certificate.
- Check 'Trust this CA to identify websites'.

- Click OK.

4.4 Intercept an HTTP Request

Steps:

- In Burp Suite, go to Proxy → Intercept tab.
- Click Intercept is off to toggle it ON.
- In Firefox, navigate to a login page (e.g., <http://localhost/dvwa/login.php>).
- Enter any credentials and click Login.
- Burp intercepts the request – it appears in the Intercept tab.
- Observe the HTTP method (POST), headers, and body parameters.

4.5 Modify the Intercepted Request

Steps:

- In the intercepted POST request, locate the username and password fields.
- Modify the values directly in the editor (e.g., change username to admin).
- Click Forward to send the modified request.
- Toggle Intercept off to allow subsequent requests through automatically.

```
# Example intercepted POST body:
username=user&password=pass&Login=Login
# Modified:
username=admin&password=admin&Login=Login
```

4.6 View HTTP History

Steps:

- Go to Proxy → HTTP History tab.
- Every request/response pair is listed here.
- Click any row to view full request and response.
- Right-click to Send to Repeater, Intruder, or Scanner.

Practical 5: Caesar Cipher

Aim: Implement the Caesar cipher substitution operation in Python

Concept: Each letter in the plaintext is shifted by a fixed number of positions in the alphabet. E.g., with shift=3: A→D, B→E, Z→C.

5.1 Algorithm

- For each character in plaintext:
- If it is a letter, shift it by key positions (mod 26).
- Preserve case (uppercase/lowercase).
- Non-alphabetic characters remain unchanged.
- Decryption uses the negative shift (or shift = 26 – key).

5.2 Python Code

```
def caesar_encrypt(plaintext, key):
    ciphertext = ''
    for char in plaintext:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            ciphertext += chr((ord(char) - base + key) % 26 + base)
        else:
            ciphertext += char
    return ciphertext

def caesar_decrypt(ciphertext, key):
    return caesar_encrypt(ciphertext, -key)

if __name__ == '__main__':
    text = input('Enter plaintext : ')
    k = int(input('Enter key (shift): '))
    enc = caesar_encrypt(text, k)
    dec = caesar_decrypt(enc, k)
    print(f'Plaintext : {text}')
    print(f'Encrypted : {enc}')
    print(f'Decrypted : {dec}')
```

5.3 Sample Output

```
Enter plaintext : Hello World
Enter key (shift): 3
Plaintext : Hello World
Encrypted : Khoor Zruog
Decrypted : Hello World
```

Practical 6: Monoalphabetic and Polyalphabetic Ciphers

Aim: Implement monoalphabetic (simple substitution) and polyalphabetic (Vigenere) cipher substitution

6.1 Monoalphabetic Cipher

Each letter of the alphabet is mapped to a unique, fixed substitute letter. The mapping is a permutation of the 26 letters.

Python Code:

```
import string

PLAIN_ALPHA = string.ascii_uppercase
```

```

CIPHER_ALPHA = 'QWERTYUIOPASDFGHJKLZXCVBNM'    # custom key alphabet

def mono_encrypt(plaintext):
    table = str.maketrans(PLAIN_ALPHA + PLAIN_ALPHA.lower(),
                          CIPHER_ALPHA + CIPHER_ALPHA.lower())
    return plaintext.translate(table)

def mono_decrypt(ciphertext):
    table = str.maketrans(CIPHER_ALPHA + CIPHER_ALPHA.lower(),
                          PLAIN_ALPHA + PLAIN_ALPHA.lower())
    return ciphertext.translate(table)

if __name__ == '__main__':
    text = input('Enter plaintext : ')
    enc  = mono_encrypt(text)
    dec  = mono_decrypt(enc)
    print(f'Cipher Alphabet: {CIPHER_ALPHA}')
    print(f'Encrypted      : {enc}')
    print(f'Decrypted      : {dec}')

```

Sample Output:

```

Enter plaintext : Hello World
Cipher Alphabet: QWERTYUIOPASDFGHJKLZXCVBNM
Encrypted      : Itssg Vgksr
Decrypted      : Hello World

```

6.2 Polyalphabetic Cipher

Uses a keyword to determine a different shift for each letter position. The keyword repeats to match the plaintext length. This prevents simple frequency analysis.

Python Code:

```

def vigenere_encrypt(plaintext, key):
    key = key.upper()
    result = ''
    ki = 0
    for char in plaintext:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            shift = ord(key[ki % len(key)]) - ord('A')
            result += chr((ord(char) - base + shift) % 26 + base)
            ki += 1
        else:
            result += char
    return result

```

```
def vigenere_decrypt(ciphertext, key):
    key = key.upper()
    result = ''
    ki = 0
    for char in ciphertext:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')
            shift = ord(key[ki % len(key)]) - ord('A')
            result += chr((ord(char) - base - shift) % 26 + base)
            ki += 1
        else:
            result += char
    return result

if __name__ == '__main__':
    text = input('Enter plaintext : ')
    key = input('Enter keyword : ')
    enc = vigenere_encrypt(text, key)
    dec = vigenere_decrypt(enc, key)
    print(f'Encrypted: {enc}')
    print(f'Decrypted: {dec}')
```

Sample Output:

```
... Enter the input : hello
Enter the input for key : 3
Encrypted: nkrru
Decrypted: hello
```

Practical 7: Playfair Cipher

Aim: Implement the Playfair cipher substitution operation

Concept: Encrypts digraphs (pairs of letters) using a 5×5 key matrix. I and J are treated as the same letter.

7.1 Rules

- Build a 5×5 matrix from the keyword (remove duplicates and unused letters; merge I/J).
- Split plaintext into digraphs; if both letters are same, insert 'X' between them; pad with 'X' if odd length.
- Same row → shift each letter one column to the right (wrap around).
- Same column → shift each letter one row down (wrap around).
- Rectangle → swap columns: each letter takes the column of the other letter in its row.

7.2 Python Code

```
def generate_matrix(key):
    key = key.upper().replace('J', 'I')
    seen = []
    for ch in key + 'ABCDEFGHIJKLMNOPQRSTUVWXYZ':
        if ch not in seen:
            seen.append(ch)
    return [seen[i*5:(i+1)*5] for i in range(5)]

def find_pos(matrix, ch):
    for r, row in enumerate(matrix):
        if ch in row:
            return r, row.index(ch)

def prepare_text(text):
    text = text.upper().replace('J', 'I').replace(' ', '')
    result = []
    i = 0
    while i < len(text):
        a = text[i]
        b = text[i+1] if i+1 < len(text) else 'X'
        if a == b:
            result.append((a, 'X'))
            i += 1
        else:
            result.append((a, b))
            i += 2
    return result

def playfair_encrypt(plaintext, key):
    matrix = generate_matrix(key)
    digraphs = prepare_text(plaintext)
    cipher = ''
    for a, b in digraphs:
        r1, c1 = find_pos(matrix, a)
        r2, c2 = find_pos(matrix, b)
        if r1 == r2:
            cipher += matrix[r1][(c1+1)%5] + matrix[r2][(c2+1)%5]
        elif c1 == c2:
            cipher += matrix[(r1+1)%5][c1] + matrix[(r2+1)%5][c2]
        else:
            cipher += matrix[r1][c2] + matrix[r2][c1]
    return cipher
```

```

def playfair_decrypt(ciphertext, key):
    matrix = generate_matrix(key)
    digraphs = [(ciphertext[i], ciphertext[i+1]) for i in range(0,
len(ciphertext), 2)]
    plain = ''
    for a, b in digraphs:
        r1,c1 = find_pos(matrix, a)
        r2,c2 = find_pos(matrix, b)
        if r1 == r2:
            plain += matrix[r1][(c1-1)%5] + matrix[r2][(c2-1)%5]
        elif c1 == c2:
            plain += matrix[(r1-1)%5][c1] + matrix[(r2-1)%5][c2]
        else:
            plain += matrix[r1][c2] + matrix[r2][c1]
    return plain

if __name__ == '__main__':
    key = input('Enter key : ')
    text = input('Enter plaintext: ')
    enc = playfair_encrypt(text, key)
    dec = playfair_decrypt(enc, key)
    mat = generate_matrix(key)
    print('\nPlayfair Matrix:')
    for row in mat: print(' '.join(row))
    print(f'\nEncrypted: {enc}')
    print(f'Decrypted: {dec}')

```

7.3 Sample Output

```

Enter key : MONARCHY
Enter plaintext: INSTRUMENTS

Playfair Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Encrypted: GATLMZCLRQTX
Decrypted: INSTRUMENTSX

```

Practical 8: Hill Cipher

Aim: Implement the Hill cipher substitution operation

Concept: A polygraphic cipher using linear algebra. The plaintext is divided into blocks of n letters, each block multiplied by an $n \times n$ key matrix (mod 26) to produce ciphertext.

8.1 Algorithm

- Choose an $n \times n$ invertible key matrix K (mod 26).
- Convert plaintext letters to numbers (A=0, B=1, ..., Z=25).
- Group into column vectors of length n (pad with 'X' if needed).
- Encrypt: $C = K \times P$ (mod 26).
- Decrypt: $P = K^{-1} \times C$ (mod 26), where K^{-1} is the modular inverse of K .

8.2 Python Code

```
import numpy as np

def mod_inverse(a, m=26):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    raise ValueError(f'{a} has no inverse mod {m}')

def matrix_mod_inv(matrix, mod=26):
    det = int(round(np.linalg.det(matrix))) % mod
    det_inv = mod_inverse(det, mod)
    n = matrix.shape[0]
    adj = (det * np.linalg.inv(matrix))
    adj = np.round(adj).astype(int) % mod
    return (det_inv * adj) % mod

def text_to_nums(text):
    return [ord(c) - ord('A') for c in text.upper() if c.isalpha()]

def nums_to_text(nums):
    return ''.join(chr(n % 26 + ord('A')) for n in nums)

def hill_encrypt(plaintext, key_matrix):
    n = key_matrix.shape[0]
    nums = text_to_nums(plaintext)
    while len(nums) % n != 0:
        nums.append(ord('X') - ord('A'))
    cipher = []
    for i in range(0, len(nums), n):
        block = np.array(nums[i:i+n])
```

```

        result = key_matrix.dot(block) % 26
        cipher.extend(result.astype(int).tolist())
    return nums_to_text(cipher)

def hill_decrypt(ciphertext, key_matrix):
    inv_key = matrix_mod_inv(key_matrix)
    return hill_encrypt(ciphertext, inv_key)

if __name__ == '__main__':
    # 2x2 key matrix
    K = np.array([[6, 24], [1, 13]])
    text = input('Enter plaintext (letters only): ').upper()
    enc = hill_encrypt(text, K)
    dec = hill_decrypt(enc, K)
    print(f'Key Matrix : \n{K}')
    print(f'Encrypted   : {enc}')
    print(f'Decrypted   : {dec}')

```

8.3 Sample Output

```

Enter plaintext (letters only): ACT
Key Matrix :
[[ 6 24]
 [ 1 13]]
Encrypted   : POH
Decrypted   : ACT

```

Practical 9: Rail Fence Cipher

Aim: Implement the Rail Fence transposition cipher

Concept: The plaintext is written diagonally across a set number of 'rails', then read off row by row to produce ciphertext.

9.1 Algorithm

- Write plaintext characters in a zigzag pattern across n rails.
- Read characters rail by rail (top to bottom) to form ciphertext.
- Decryption reconstructs the zigzag grid and reads diagonally.

9.2 Python Code

```

def rail_fence_encrypt(plaintext, rails):
    fence = [[] for _ in range(rails)]
    rail = 0
    direction = 1
    for char in plaintext:

```

```

        fence[rail].append(char)
    if rail == 0:
        direction = 1
    elif rail == rails - 1:
        direction = -1
    rail += direction
    return ''.join(''.join(r) for r in fence)

def rail_fence_decrypt(ciphertext, rails):
    n = len(ciphertext)
    pattern = []
    rail = 0
    direction = 1
    for i in range(n):
        pattern.append(rail)
        if rail == 0:
            direction = 1
        elif rail == rails - 1:
            direction = -1
        rail += direction
    indices = sorted(range(n), key=lambda x: pattern[x])
    result = [''] * n
    for i, ch in zip(indices, ciphertext):
        result[i] = ch
    return ''.join(result)

if __name__ == '__main__':
    text = input('Enter plaintext: ')
    rails = int(input('Enter number of rails: '))
    enc = rail_fence_encrypt(text, rails)
    dec = rail_fence_decrypt(enc, rails)
    print(f'Encrypted: {enc}')
    print(f'Decrypted: {dec}')

```

9.3 Sample Output

```

Enter plaintext: WEAREDISCOVEREDFLEEAOAH
Enter number of rails: 3

Rail visualization:
Rail 0: W . . . E . . . I . . . V . . . E . . . A . . .
Rail 1: . E . R . D . S . O . E . E . F . E . A . O . H
Rail 2: . . A . . . C . . . D . . . L . . . H . . .

Encrypted: WEIVEDAERDSOEFEAOHACL
Decrypted: WEAREDISCOVEREDFLEEAOAH

```

Practical 10: Row Transposition Cipher

Aim: Implement the Row Transposition (Columnar Transposition) cipher

Concept: Plaintext is written in rows of fixed width, then columns are reordered according to the alphabetical order of the key to produce ciphertext.

10.1 Algorithm

- Write plaintext row by row in a grid of width = len(key). Pad with 'X' if needed.
- Determine column order by sorting key letters alphabetically.
- Read off columns in the sorted key order to produce ciphertext.
- Decryption fills columns in sorted order, then reads rows.

10.2 Python Code

```
import math

def row_transposition_encrypt(plaintext, key):
    cols = len(key)
    padded = plaintext.ljust(math.ceil(len(plaintext)/cols)*cols, 'X')
    rows = [padded[i:i+cols] for i in range(0, len(padded), cols)]
    order = sorted(range(cols), key=lambda x: key[x])
    cipher = ''
    for col in order:
        cipher += ''.join(row[col] for row in rows)
    return cipher

def row_transposition_decrypt(ciphertext, key):
    cols = len(key)
    n_rows = len(ciphertext) // cols
    order = sorted(range(cols), key=lambda x: key[x])
    col_len = n_rows
    columns = {}
    idx = 0
    for col in order:
        columns[col] = list(ciphertext[idx:idx+col_len])
        idx += col_len
    plain = ''
    for r in range(n_rows):
        for c in range(cols):
            plain += columns[c][r]
    return plain

if __name__ == '__main__':
    key = input('Enter key (e.g. SECURITY): ').upper()
    text = input('Enter plaintext : ').upper().replace(' ', '')
```

```
enc = row_transposition_encrypt(text, key)
dec = row_transposition_decrypt(enc, key)
print(f'Encrypted: {enc}')
print(f'Decrypted: {dec}')
```

10.3 Sample Output

```
Enter key (e.g. SECURITY): SECURITY
Enter plaintext           : WEAREDISCOVEREDFLEEAOAH

Key column order: E=1 C=0 I=3 R=5 T=6 U=7 Y=8 S=4

Encrypted: EVLNACDTESEAROFODEECWIREE
Decrypted: WEAREDISCOVEREDFLEEAOAHX
```

Practical 11: Product Cipher

Aim: Implement the Product cipher transposition operation

Concept: A product cipher combines two or more cipher operations sequentially. The plaintext is first encrypted with one cipher, and the result is then encrypted with another. This increases security by adding multiple layers of transformation. A common combination: Caesar cipher (substitution) followed by Rail Fence cipher (transposition).

11.1 Algorithm

- Step 1 – Apply Caesar cipher (substitution) with a given shift.
- Step 2 – Apply Rail Fence cipher (transposition) on the output of Step 1.
- Decryption is the reverse: first Rail Fence decrypt, then Caesar decrypt.
- The combined cipher is much harder to break than either alone.

11.2 Python Code

```
# === Caesar Cipher ===
def caesar_encrypt(text, shift):
    result = ''
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result += chr((ord(ch) - base + shift) % 26 + base)
        else:
            result += ch
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)
```

```

# === Rail Fence Cipher ===
def rail_fence_encrypt(text, rails):
    fence = [[] for _ in range(rails)]
    rail, direction = 0, 1
    for ch in text:
        fence[rail].append(ch)
        if rail == 0:            direction = 1
        elif rail == rails - 1: direction = -1
        rail += direction
    return ''.join(''.join(r) for r in fence)

def rail_fence_decrypt(text, rails):
    n, pattern = len(text), []
    rail, direction = 0, 1
    for _ in range(n):
        pattern.append(rail)
        if rail == 0:            direction = 1
        elif rail == rails - 1: direction = -1
        rail += direction
    indices = sorted(range(n), key=lambda x: pattern[x])
    result = [''] * n
    for i, ch in zip(indices, text):
        result[i] = ch
    return ''.join(result)

# === Product Cipher ===
def product_encrypt(plaintext, caesar_shift, rf_rails):
    after_sub = caesar_encrypt(plaintext, caesar_shift)
    after_trans = rail_fence_encrypt(after_sub, rf_rails)
    return after_sub, after_trans

def product_decrypt(ciphertext, caesar_shift, rf_rails):
    after_rail = rail_fence_decrypt(ciphertext, rf_rails)
    after_caesar = caesar_decrypt(after_rail, caesar_shift)
    return after_rail, after_caesar

if __name__ == '__main__':
    text = input('Enter plaintext      : ')
    shift = int(input('Enter Caesar shift : '))
    rails = int(input('Enter Rail Fence rails: '))

    sub_enc, prod_enc = product_encrypt(text, shift, rails)
    rail_dec, prod_dec = product_decrypt(prod_enc, shift, rails)

```

```
print(f'\nOriginal      : {text}')
```

```
print(f'After Caesar Enc  : {sub_enc}')
```

```
print(f'Final Ciphertext : {prod_enc}')
```

```
print(f'--- Decryption ---')
```

```
print(f'After Rail Dec    : {rail_dec}')
```

```
print(f'Final Plaintext   : {prod_dec}')
```

11.3 Sample Output

```
Enter plaintext      : HELLO WORLD
Enter Caesar shift   : 3
Enter Rail Fence rails: 2

Original            : HELLO WORLD
After Caesar Enc     : KHOOR ZRUOG
Final Ciphertext     : KORZU HORZG
--- Decryption ---
After Rail Dec       : KHOOR ZRUOG
Final Plaintext      : HELLO WORLD
```